

Readme Honeybot

HoneyBot: AWS EC2 SSH Honeypot & Threat Intelligence System

Document Control

- **Author:** Nic D
- **Subject:** Cloud Infrastructure Hardening & Network Threat Analysis
- **Classification:** TLP:CLEAR (Publicly distributable)
- **Date:** July 14, 2026

1. Executive Summary

This technical report documents the step-by-step deployment, architectural evolution, and automated forensic results of **HoneyBot**, a custom-built, low-interaction SSH honeypot. Deployed within an Amazon Web Services (AWS) EC2 sandbox, the system was engineered to attract, log, analyze, and mitigate malicious automated scans, botnets, and brute-force credential stuffing attacks originating from the public IPv4 space.

The project demonstrates an iterative engineering lifecycle, migrating from a basic banner-level socket listener (**Phase 1**) to an interactive, protocol-level emulator featuring automated firewall containment (**Phase 2**).

2. Infrastructure & Network Architecture

2.1 Server Specifications

- **Cloud Provider:** Amazon Web Services (AWS)
- **Region:** US East (Ohio) / `us-east-2`
- **Operating System:** Ubuntu 24.04 LTS (GNU/Linux 6.17.0-1017-aws x86_64)
- **Security Group ID:** `sg-0f23696c2b3b84d5`

2.2 Split-Port Configuration

To expose the default SSH bait port while preserving uninterrupted administrative control, we implemented a strict **split-port network boundary**:

- **Port 22 (The Bait):** Configured as completely open (`0.0.0.0/0` and `::/0`) within the AWS Security Group. Any traffic hitting this standard port is systematically routed to the active

Honeybot engine.

- **Port 2222 (Administrative Gateway):** The production OpenSSH daemon (`sshd`) was migrated to this high port in `/etc/ssh/sshd_config` . Ingress rules restrict traffic strictly to the administrator's authorized IP address, preventing external scanning and unauthorized access.

3. Iterative Development Cycle

Phase 1: Banner-Level Trap (`honeybot.sh`)

The initial prototype used a lightweight Bash script wrapping standard Unix networking utilities (`netcat`).

- **Design:** When an external client initiates a TCP handshake on port 22, the script injects a simulated OpenSSH version banner (`SSH-2.0-OpenSSH_8.9p1`) to mimic a vulnerable Linux server, logging the connection metadata.
- **Optimization:** To resolve high CPU utilization caused by the default `netcat-openbsd` package in modern Ubuntu environments (which strips the execution flags), the pipeline was stabilized using dynamic redirection streams and proactive socket clearing via `fuser` .
- **Technical Limitation:** This model only records network-level connections. Because it does not complete the cryptographic SSH key exchange, it is structurally unable to capture authentication details or payload credentials.

 **Codebase Reference** The stable implementation of this phase is stored in the file `honeybot.sh` within the repository root directory.

Phase 2: Protocol-Level Emulation (`honeybot.py`)

To capture deep interactive metrics, the system was completely rewritten in Python utilizing the `Paramiko` library, effectively simulating an interactive SSH server layer.

- **Credential Harvesting:** The emulator successfully negotiates the complex Diffie-Hellman cryptographic transport layer and host-key verification. Once the channel is active, it captures plaintext usernames and passwords, writes them directly to log, and terminates the connection with an `AUTH_FAILED` response.
- **Active Mitigation Engine:** An automated sliding-window brute-force counter was programmed. If an IP address exceeds a configurable threshold, the server executes an administrative OS-level ban command:

$$\text{Threshold} \geq 5 \text{ login attempts within } T_{\text{window}} = 60 \text{ s}$$

⚠️ **Active Containment** Upon triggering a ban, the system automatically spawns a subprocess to append a DROP rule to the Linux firewall: `sudo iptables -A INPUT -s <IP> -j DROP` This dynamically isolates persistent threat actors at the network boundary without taking external aggressive actions.

- **Structured Analytics:** Telemetry is written in structured JSON Lines (`honeybot.jsonl`) to enable fast ingestion by SIEMs or custom analytics engines.

4. Empirical Threat Data & Analysis

4.1 Phase 1 Connections (Passive Metadata Collection)

Within minutes of being deployed on the public internet, the honeypot detected active automated probes.

Timestamp (UTC)	Attacking IP	Domain Pointer (rDNS/PTR)	Registered Organization
04:04:41	<i>Dynamic</i>	<code>customer.nywyywl.isp.starlink.com</code>	SpaceX / Starlink
04:04:55	<code>65.81.41.31</code>	<code>65.81.41.31.ip4.feromedia.eu</code>	AT&T Enterprises
04:27:37	<code>86.107.159.156</code>	<i>None</i>	RIPE Network Block
04:38:34	<code>112.195.132.66</code>	<code>112.195.132.66.censys-scanner.com</code>	Censys Inc.
04:40:55	<code>184.178.172.10</code>	<code>wsip-184-178-172-10.rn.hr.cox.net</code>	Cox Communications

4.2 Section 5.3: Case Study - The Reverse DNS Deception

During the initial analysis, a suspicious connection originated from the IP address `65.81.41.31`.

⚡ **The Attribution Mismatch**

- **Reported PTR Record (rDNS):** 65.81.41.31.ip4.feromedia.eu → Points to a European hosting provider (Feromedia).
- **Deep Registry Interrogation (WHOIS Map):**
 - **NetRange:** 65.64.0.0 - 65.91.255.255
 - **NetName:** ATT-INTERNET-SERVICES
 - **Organization:** AT&T Enterprises, LLC (Dallas, Texas, USA)

Forensic Takeaways

1. **The Attribution Trap:** Threat actors rely on stale, misconfigured, or intentionally altered Reverse DNS pointer records to deceive automated security tools. Simple automated abuse reporting without IP verification would have misdirected the complaint to a European host.
2. **Conscription into IoT Botnets:** The underlying attacker is not a hosting server, but an ordinary residential or commercial router in Dallas, Texas. This node was compromised by automated malware (such as a Mirai variant) and conscripted into scanning the IPv4 space for other open SSH services.

4.3 Phase 2 Connections (Active Credential Analysis)

Upgrading the honeypot to Phase 2 enabled the deep capturing of attacker payloads. A persistent Dutch-hosted threat actor (91.92.40.36) launched a rapid dictionary attack on our instance, allowing the mitigation engine to cleanly trigger an automated `iptables` ban.

📘 **Sliding-Window Block Event At the 5th credential injection within a span of 18 seconds (well under our 60s threshold), HoneyBot triggered a ban on 91.92.40.36 . No further connection attempts were registered, validating the proactive firewall mitigation logic.**

Captured Credentials Payload

Timestamp (UTC)	Attacker IP	Submitted Username	Submitted Password	Target Vector / Pattern Analysis
16:39:55	91.92.40.36	root	ankurkudintzi	Dictionary exploit targeting default administrator accounts
16:42:13	91.92.40.36	brad	brad	Username-as-Password (exploits weak default setups)

Timestamp (UTC)	Attacker IP	Submitted Username	Submitted Password	Target Vector / Pattern Analysis
16:42:24	91.92.40.36	winston	winston	Username-as-Password (exploits weak default setups)
16:42:31	91.92.40.36	ui	ui	Target scanning aimed at Ubiquiti UniFi Controllers
16:42:38	91.92.40.36	kafka	kafka	Target scanning aimed at Apache Kafka Portals
16:42:46	91.92.40.36	root	!@#\$\$%^	Hardcoded system brute-force / Password bypass patterns

The targeted authentication queries for accounts like `ui` and `kafka` reveal that modern automated botnets do not just perform blind random brute-forcing; they carry highly focused payloads trying to identify specific exposed enterprise cloud controllers and IoT setups.

5. Deployment Instructions

To set up and run this honeypot framework on your cloud instance:

Step 1: Secure Production SSH

Migrate the legitimate server's admin SSH daemon to a high port:

```
sudo nano /etc/ssh/sshd_config
# Modify "Port 22" to "Port 2222"
sudo systemctl restart sshd
```

(Verify your AWS Security Group permits TCP ingress traffic on port 2222 for your admin IP before disconnecting).

Step 2: Establish the Python Environment

Install Paramiko inside your environment:

```
pip install paramiko
```

Step 3: Launch the Active Server

Start the Python script as a background service:

```
sudo nohup python3 honeybot.py > /dev/null 2>&1 &
```

Step 4: Run the Telemetry Analyzer

Run the analytics framework to process structural telemetry in real-time:

```
python3 honeybot_analysis.py
```

6. Project Conclusions

1. **Exposure Velocity:** Newly allocated cloud instances receive malicious probe traffic within minutes. This emphasizes that simply leaving a standard port exposed to the internet is an immediate security risk passive network exposure is an immediate threat vector in modern cloud architectures.
2. **Infrastructure Exploitation:** Hostile scanning patterns originate largely from compromised consumer devices (such as residential routers) and low-tier rented hosting servers rather than advanced, professional hacker setups structured, specialized offensive infrastructures.
3. **Low-Interaction Safety:** By simulating a real SSH server, we Moving to an interactive emulating model allowed us to harvest deep threat intelligence (targeted applications, passwords, threat origins) while preventing attackers from obtaining actual shells, preserving complete system integrity.
4. **Proactive Defense (Auditing Blind Spots):** By capturing the exact usernames and passwords attackers are prioritizing (such as targeted scans for `ui` or `kafka`), organizations can cross-reference this intelligence against their own infrastructure. This allows security teams to proactively audit and eliminate default or weak credentials *before* they are exploited on real production systems.
5. **High-Fidelity "Early Warning" System:** Instead of relying solely on expensive, generic threat intelligence feeds, deploying a lightweight honeypot adjacent to production servers provides a highly accurate, custom list of malicious IPs currently active in the organization's specific cloud neighborhood. These IPs can then be automatically fed into corporate firewalls to block attackers before they ever touch a real, valuable server.